

Self Learning 2

1. Case Study Overview

The Healthcare Management System is a software system designed to manage the activities of a hospital or healthcare centre. It provides a centralized platform for managing patients, doctors, appointments, medical records, prescriptions, billing and payments.

The case study demonstrates how object-oriented analysis and design can be applied to a real-world domain. UML diagrams are used to represent the structure and behaviour of the system. SOLID principles, refactoring and Domain-Driven Design (DDD) concepts are used to improve maintainability, flexibility and scalability.

2. StarUML and UML Diagrams

StarUML is a UML modeling tool used to create visual models of software systems. For the Healthcare Management System, the main actors are Patient, Doctor, Receptionist, Administrator, and Billing Staff. The system manages registration, appointments, consultation, prescriptions, medical records, billing, and payment.

2.1 Use Case Diagram

A use case diagram shows what users or external actors expect from the system. It focuses on system functionality rather than implementation details.

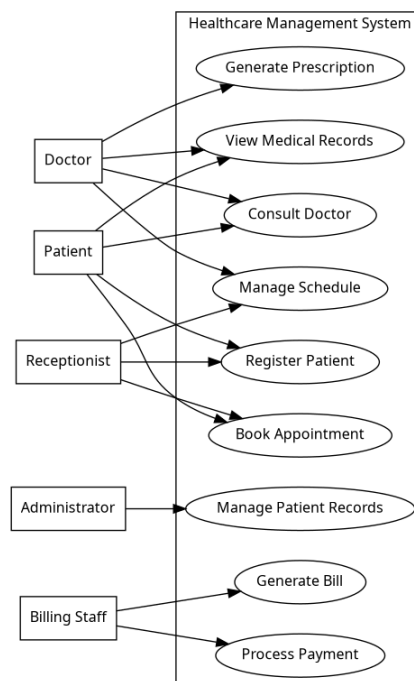


Figure 1: Use Case Diagram – Healthcare Management System

How to create it in StarUML:

1. Create a new UML project and add a Use Case Diagram.
2. Add a System Boundary named Healthcare Management System.
3. Add actors such as Patient, Doctor, Receptionist, Administrator, and Billing Staff.
4. Add use cases such as Register Patient, Book Appointment, Consult Doctor, Generate Prescription, Generate Bill, and Process Payment.
5. Connect each actor to the use cases they participate in using Association relationships.

2.2 Class Diagram

A class diagram represents the static structure of the system. It shows classes, attributes, operations, and relationships between classes.

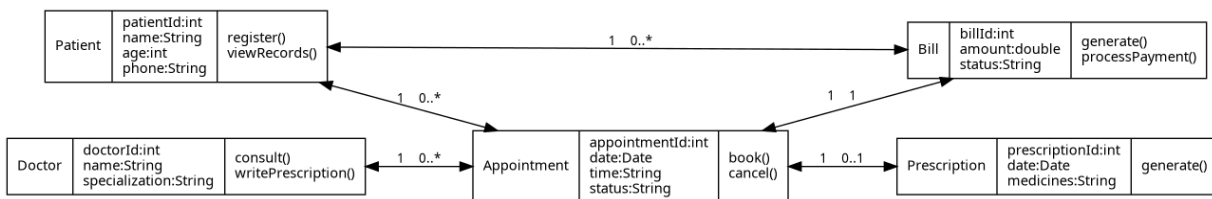


Figure 2: Class Diagram – Healthcare Management System

Important classes include Patient, Doctor, Appointment, Prescription, and Bill. For example, one patient can have many appointments, while an appointment is associated with one doctor. An appointment may produce a prescription and a bill.

How to create it in StarUML:

1. Create a Class Diagram.
2. Add the required classes and enter attributes and operations in each class.
3. Connect classes with Association relationships.
4. Set multiplicities such as 1, 0..1, and 0..* where required.
5. Use Generalization for inheritance and the appropriate diamond notation for aggregation or composition.

2.3 Sequence Diagram

A sequence diagram shows the order in which objects or system components exchange messages over time. It is useful for understanding a particular scenario such as booking and completing a patient appointment.

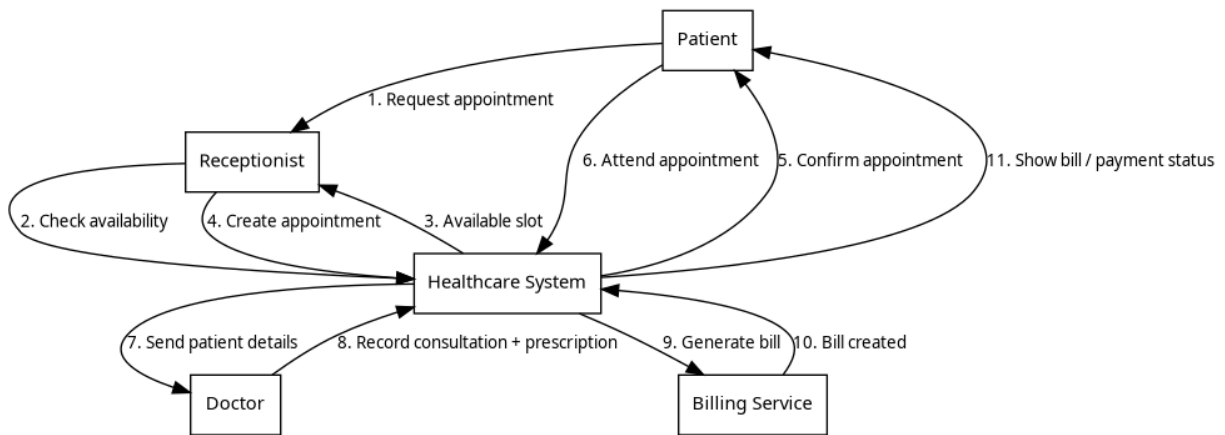


Figure 3: Sequence Diagram – Appointment and Consultation Flow

The sequence begins when the patient requests an appointment. The receptionist or application checks availability, creates the appointment, confirms it, sends patient details to the doctor, records the consultation, and finally generates the bill.

2.4 Activity Diagram

An activity diagram models the workflow of a process. It is suitable for showing decisions, loops, parallel activities, and the overall flow from start to completion.

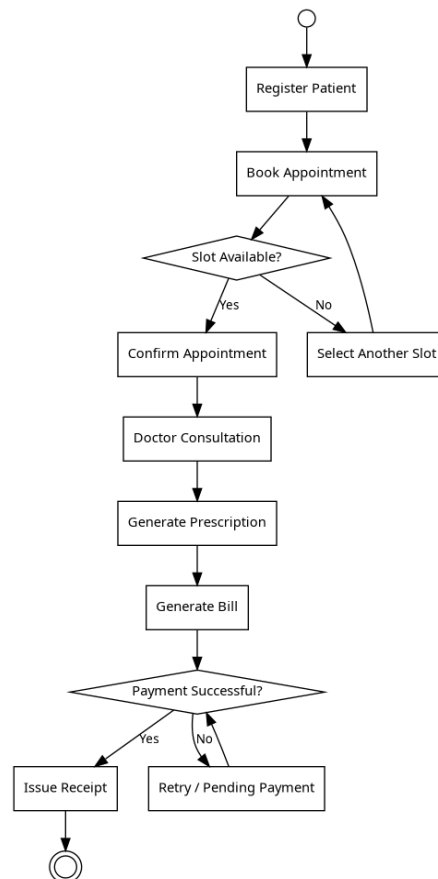


Figure 4: Activity Diagram – Healthcare Service Workflow

The workflow includes patient registration, appointment booking, availability checking, consultation, prescription generation, billing, and payment. The decision nodes demonstrate what happens when an appointment slot is unavailable or payment fails.

2.5 Component / Deployment Diagram

A component or deployment view shows the major software components and the physical or execution environment in which they operate. The following diagram combines the practical deployment view for a web-based healthcare application.

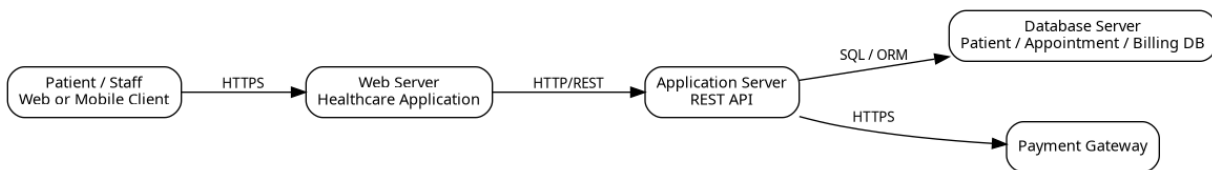


Figure 5: Component / Deployment View

A typical deployment consists of a patient/staff client, web server, application server, database server, and external payment gateway. Communication can use HTTPS and REST APIs, while the application server communicates with the database through SQL or an ORM.

2.6 UML Relationships

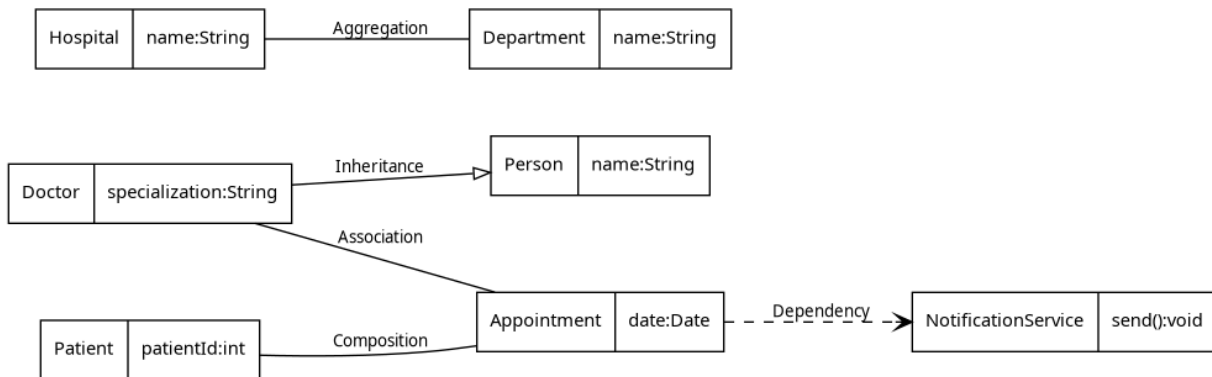


Figure 6: Examples of UML Relationships

Relationship	Meaning	Healthcare Example
Association	A general structural connection between classes.	Doctor — Appointment
Inheritance / Generalization	A child class inherits features from a parent class.	Doctor inherits from Person
Aggregation	A whole contains parts, but the parts can exist independently.	Hospital contains Departments
Composition	A strong whole-part relationship; the part depends on the whole.	Patient contains Appointment records in a model where the record lifecycle is owned by Patient
Dependency	One class temporarily uses another class.	Appointment uses NotificationService

In StarUML, these relationships can be selected from the UML toolbox and connected between the appropriate model elements. Multiplicity should be added when the number of related objects is important.

3. Applying SOLID Principles

SOLID is a group of five object-oriented design principles that improve maintainability, flexibility, testability, and separation of responsibilities. The following examples use healthcare and payment-related scenarios.

3.1 Single Responsibility Principle (SRP)

Principle: A class should have one primary responsibility and therefore one main reason to change.

Bad design: A single BankAccount class performs account operations, creates statements, sends emails, and prints receipts. These responsibilities belong to different concerns.



Figure 7: SRP – Before and After Refactoring

Problem identified: The class becomes large, difficult to test, and changes in reporting or notification requirements can affect the banking logic.

Refactoring: Keep BankAccount responsible for account operations. Move statement generation to StatementService and notifications to NotificationService. This separates business rules from supporting services.

Applied principle: Single Responsibility Principle.

3.2 Open/Closed Principle (OCP)

Principle: Software entities should be open for extension but closed for modification.

Bad design: Checkout contains a long if/else or switch statement for Card, UPI, Cash, and every future payment type.

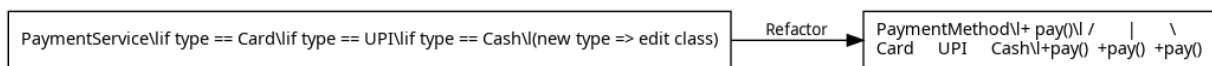


Figure 8: OCP – Payment Method Refactoring

Problem identified: Every new payment method requires modifying the existing Checkout or PaymentService class, increasing regression risk.

Refactoring: Define a PaymentMethod abstraction with a pay() operation. CardPayment, UPIPayment, and CashPayment implement that abstraction. A new method can then be added as a new class.

Applied principle: Open/Closed Principle.

3.3 Liskov Substitution Principle (LSP)

Principle: Objects of a subclass should be usable wherever objects of the superclass are expected without breaking the correctness of the program.

Bad design: A generic Bird class declares fly(). Penguin inherits Bird but cannot logically fly. Calling fly() on Penguin violates the expected behavior of the parent abstraction.

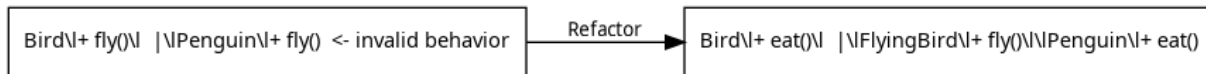


Figure 9: LSP – Separating Flying Behavior

Problem identified: The inheritance hierarchy promises behavior that some subclasses cannot support.

Refactoring: Keep common behavior in Bird, such as eat(). Put fly() in a FlyingBird abstraction and use it only for birds that can fly.

Applied principle: Liskov Substitution Principle.

3.4 Interface Segregation Principle (ISP)

Principle: Clients should not be forced to depend on methods they do not use.

Bad design: A single Worker interface contains print(), scan(), and fax(). A basic printer that only prints is forced to implement scan() and fax().

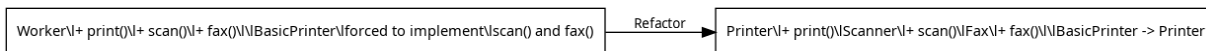


Figure 10: ISP – Splitting a Large Interface

Problem identified: Unnecessary methods lead to dummy implementations and tightly coupled interfaces.

Refactoring: Split the large interface into smaller Printer, Scanner, and Fax interfaces. A device implements only the capabilities it actually provides.

Applied principle: Interface Segregation Principle.

3.5 Dependency Inversion Principle (DIP)

Principle: High-level modules should depend on abstractions rather than concrete low-level implementations.

Bad design: Checkout directly creates and uses StripePayment. Changing the payment provider requires changing Checkout.



Figure 11: DIP – Depending on a Payment Abstraction

Problem identified: Checkout is tightly coupled to one payment provider, making testing and replacement harder.

Refactoring: Introduce a PaymentGateway interface. StripePayment, another provider, or a test double can implement it. Checkout receives the abstraction through dependency injection.

Applied principle: Dependency Inversion Principle.

SOLID Summary

Principle	Main Idea	Typical Benefit
SRP	One class, one primary responsibility.	Less complexity and easier maintenance.
OCP	Extend behavior without repeatedly modifying stable code.	Safer addition of new features.
LSP	Subtypes must preserve the expected behavior of their base abstraction.	Correct and predictable polymorphism.
ISP	Prefer small, focused interfaces.	Less unnecessary coupling.
DIP	Depend on abstractions, not concrete implementations.	Flexible architecture and easier testing.

4. Refactoring and Code Smells

A code smell is a symptom of a design or implementation problem. A smell does not necessarily mean the program is incorrect, but it indicates that the code may become difficult to understand, test, or modify. Refactoring changes the internal structure without changing the intended external behavior.

4.1 Long Method

Problem: One method performs registration, appointment validation, billing, notification, and report generation.

Refactoring technique: Extract Method. Break the long method into small methods with meaningful names.

Before:

```
processPatient() {  
    registerPatient();  
    validateAppointment();  
    calculateBill();  
    sendNotification();  
    generateReport();  
}
```

After:

```

processPatient() {
    registerPatient();
    validateAppointment();
    generateBill();
    notifyPatient();
}

```

4.2 Large / God Class

Problem: A single HealthcareManager class contains patient management, doctor scheduling, billing, authentication, reporting, and notification logic.

Refactoring technique: Extract Class. Create focused classes such as PatientService, AppointmentService, BillingService, AuthenticationService, and NotificationService.

4.3 Duplicate Code

Problem: The same bill calculation or validation logic is copied into multiple classes.

Refactoring technique: Extract Method or Extract Class. Place the common logic in one reusable method or service and call it from the required locations.

4.4 Long Parameter List

Problem: A method such as bookAppointment(patientId, doctorId, date, time, reason, phone, insuranceNo, emergencyFlag) becomes difficult to call and maintain.

Refactoring technique: Introduce Parameter Object. Group related data into an AppointmentRequest object.

```

AppointmentRequest request =
    new AppointmentRequest(patientId, doctorId, date, time, reason);

bookAppointment(request);

```

4.5 Tight Coupling

Problem: AppointmentService directly creates DatabaseConnection and EmailSender objects. Replacing the database or notification provider requires editing the business logic.

Refactoring technique: Dependency Injection. Pass repository and notification abstractions into AppointmentService.

```

AppointmentService( AppointmentRepository repo,
    NotificationService notifier )

```

Refactoring Summary

Code Smell	Problem	Refactoring Technique
Long Method	Too much logic in one method.	Extract Method
Large/God Class	Too many responsibilities.	Extract Class
Duplicate Code	Same logic repeated.	Extract Method / Class

Long Parameter List	Method is hard to call and understand.	Parameter Object
Tight Coupling	Classes depend directly on concrete implementations.	Dependency Injection / abstractions

5. Domain-Driven Design (DDD) Basics

Domain-Driven Design focuses software design on the business domain and its important concepts. For healthcare, the domain contains patients, doctors, appointments, clinical records, prescriptions, and billing rules. DDD helps keep business language and software structure aligned.

5.1 Entities

Entities are domain objects identified by a stable identity over time. Their attributes may change, but their identity remains meaningful.

- Patient – identified by patientId.
- Doctor – identified by doctorId.
- Appointment – identified by appointmentId.
- Prescription – identified by prescriptionId.

5.2 Value Objects

Value Objects describe characteristics and normally do not have an independent identity. They are compared by their values.

- Address – street, city, state, postal code.
- PhoneNumber – country code and number.
- Money – amount and currency.
- AppointmentTime – date and time represented as a meaningful domain value.

5.3 Aggregates

An Aggregate is a consistency boundary containing related entities and value objects. An Aggregate Root is the main object through which the outside world accesses the aggregate.

Example: Appointment can be modeled as an aggregate root that controls appointment status, scheduled time, assigned doctor, and related appointment rules. External code should use Appointment's domain operations instead of directly changing internal state.

5.4 Repositories

A Repository provides a domain-oriented interface for retrieving and storing aggregates. It hides database details from domain logic.

```
interface AppointmentRepository {
    Appointment findById(int id);
    void save(Appointment appointment);
}
```

5.5 Domain Services

A Domain Service contains business logic that does not naturally belong to one entity or value object. Examples include AppointmentBookingService, BillingService, and InsuranceEligibilityService.

5.6 Bounded Contexts

A Bounded Context defines a boundary within which a particular domain model and terminology are consistent. Large healthcare systems can be separated into contexts such as Patient Management, Scheduling, Clinical Care, and Billing.

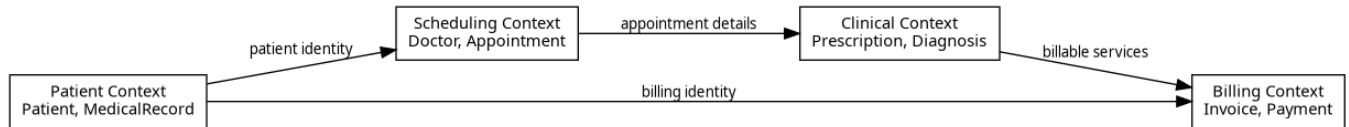


Figure 12: Healthcare DDD Bounded Contexts

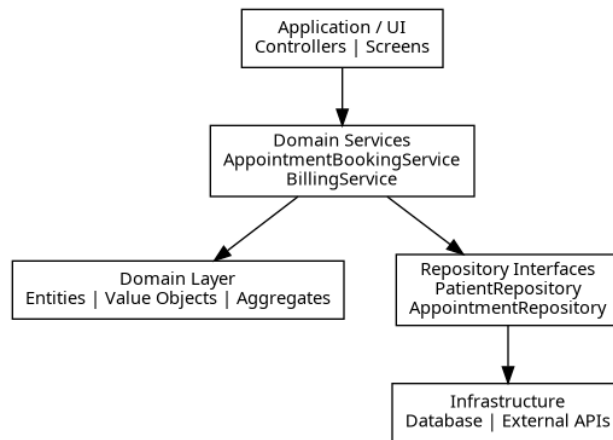


Figure 13: Simplified DDD Layered View

DDD Building Block	Healthcare Example	Purpose
Entity	Patient, Doctor, Appointment	Identity-based domain object.
Value Object	Address, Money, PhoneNumber	Describes a value without independent identity.
Aggregate	Appointment aggregate	Defines a consistency boundary.
Repository	AppointmentRepository	Loads and saves aggregates.
Domain Service	AppointmentBookingService	Encapsulates domain logic spanning objects.
Bounded Context	Scheduling, Clinical, Billing	Defines a clear model and terminology boundary.

6. Object-Oriented Design vs Structured Design

The same Hospital Management workflow can be designed using object-oriented or structured approaches. The business flow is: Register Patient → Book Appointment → Consult Doctor → Generate Prescription → Generate Bill.

6.1 Object-Oriented Design

Object-oriented design models the system as interacting objects that combine state and behavior. Each major domain concept is represented by a class.

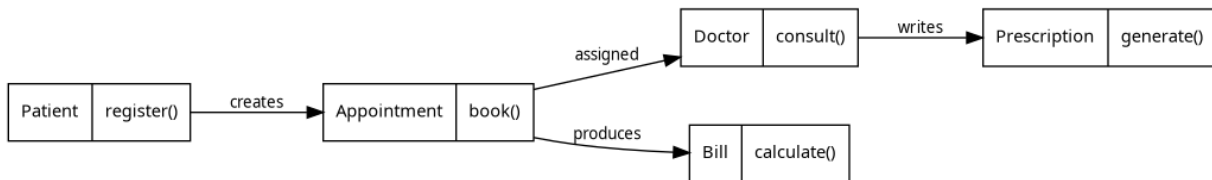


Figure 14: Object-Oriented Design

Example responsibilities:

- Patient.register() manages patient-related behavior.
- Appointment.book() manages appointment behavior.
- Doctor.consult() represents consultation behavior.
- Prescription.generate() creates prescription information.
- Bill.calculate() performs bill-related behavior.

6.2 Structured / Procedural Design

Structured design focuses on procedures or functions and the data passed between them. The workflow is commonly decomposed into functions such as register_patient(), book_appointment(), consult_doctor(), generate_prescription(), and generate_bill().

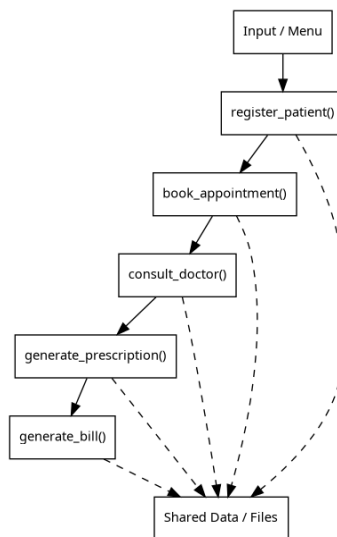


Figure 15: Structured / Procedural Design

6.3 Comparison

Aspect	Object-Oriented Design	Structured Design
Main unit	Objects / classes	Functions / procedures
Data and behavior	Usually encapsulated together	Often separated
Reuse	Inheritance, composition, polymorphism	Function reuse and modules
Change management	Changes can be localized to classes	Changes may affect shared data and functions
Best suited for	Complex domains with many interacting concepts	Algorithmic and process-oriented problems
Healthcare example	Patient, Doctor, Appointment, Bill classes	register_patient(), book_appointment(), generate_bill()

For a large healthcare application, object-oriented design is generally more suitable because the domain contains many entities with state, behavior, and relationships. Structured design remains useful for small, sequential, or strongly algorithmic processes.

7. Conclusion

This self-learning study demonstrates how software design techniques can be applied systematically to a Healthcare Management System. StarUML and UML diagrams provide visual models of system behavior, structure, interactions, and deployment. SOLID principles improve maintainability by separating responsibilities, supporting extension, protecting substitutability, reducing interface dependencies, and depending on abstractions.

Refactoring helps improve poorly structured code by addressing smells such as Long Method, God Class, Duplicate Code, Long Parameter List, and Tight Coupling. DDD provides a domain-focused way to identify entities, value objects, aggregates, repositories, services, and bounded contexts. Finally, comparing object-oriented and structured designs shows how the choice of design approach affects modularity, data encapsulation, reuse, and change management.

Overall, the combination of UML modeling, SOLID, refactoring, and DDD provides a strong foundation for designing maintainable and scalable software systems.